Introduction to Flutter UI Design

Lab Objectives

After completing this lab, the student will be able to:

- Understand the fundamentals of Flutter and its advantages in mobile development
- Explain Flutter's widget-based architecture
- Differentiate between Stateless and Stateful widgets
- Use common Flutter UI widgets to build simple interfaces
- Create multiple screens and navigate between them using Flutter routing

Tools Required

- Flutter SDK
- Dart SDK (included with Flutter)
- Android Studio or Visual Studio Code
- Android Emulator or Physical Android Device
- Basic knowledge of programming concepts

Introduction to Flutter

1. What is Flutter and Why to Use It

Flutter is an open-source UI software development kit developed by Google for building cross-platform applications from a single codebase. Flutter allows developers to create high-performance mobile applications for Android and iOS using the Dart programming language.

Why Flutter:

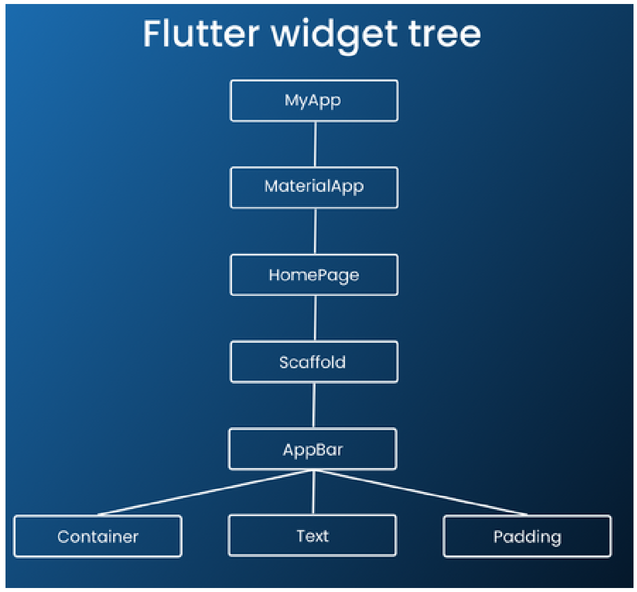
- Single codebase for multiple platforms
- Fast development using Hot Reload
- Rich set of customizable widgets
- High performance close to native apps
- Strong community and industry adoption

2. Flutter Widget Architecture

Flutter follows a **widget-based architecture**, where **everything in Flutter is a widget**. Widgets describe how the UI should look and behave.

Everything is a Widget

- Text, buttons, layouts, screens — all are widgets
- Widgets are immutable and form a **widget tree**



MaterialApp

- Root widget of a Flutter application
- Provides app-level configuration like theme and routing

```
MaterialApp(
  home: HomeScreen(),
);
```

Scaffold

- Provides basic visual structure for a screen, Components include:
    - AppBar
    - body
    - BottomNavigationBar

```
Scaffold(
  appBar: AppBar(title: Text("Home")),
  body: Center(child: Text("Welcome")),
);
```

LAB

Creating and Running a Flutter Application

1. Open terminal and run `flutter create my_app`
2. Navigate to the project directory: `cd my_app`
3. Run the application: `flutter run`
4. The default Flutter counter app will appear on the emulator/device

Task 1: Create a Simple Registration (Login) Screen

Objective:
Design a login screen that accepts user credentials.

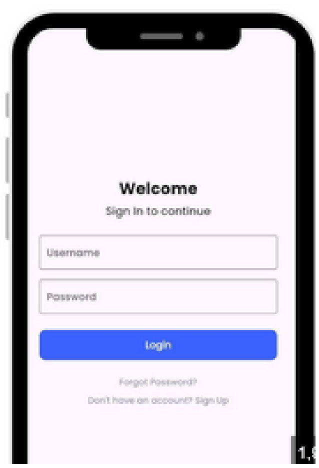
Requirements:

- Use Column for layout
- Use TextField for username and password
- Use ElevatedButton for login action
- Display appropriate icons

Expected UI Components:

- AppBar with title “Login”
- Two TextFields (Username, Password)
- Login Button

Expected UI Output :

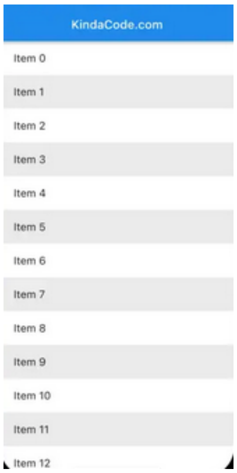


```dart
import 'package:flutter/material.dart';

class LoginScreen extends StatelessWidget {
  const LoginScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: Colors.white,
      body: Padding(
        padding: const EdgeInsets.all(24.0),
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Text(
              'Welcome',
              style: TextStyle(fontSize: 28, fontWeight: FontWeight.bold),
            ),
            const SizedBox(height: 8),
            const Text(
              'Sign in to continue',
              style: TextStyle(fontSize: 14, color: Colors.grey),
            ),
            const SizedBox(height: 40),
            TextField(
              decoration: InputDecoration(
                labelText: 'Username',
                border: OutlineInputBorder(
                  borderRadius: BorderRadius.circular(8),
                ),
              ),
            ),
            const SizedBox(height: 16),
            TextField(
              obscureText: true,
              decoration: InputDecoration(
                labelText: 'Password',
                border: OutlineInputBorder(
                  borderRadius: BorderRadius.circular(8),
                ),
              ),
            ),
            const SizedBox(height: 24),
            SizedBox(
              width: double.infinity,
              child: ElevatedButton(
                onPressed: () {},
                style: ElevatedButton.styleFrom(
                  backgroundColor: Colors.blue,
                  padding: const EdgeInsets.all(16),
                  shape: RoundedRectangleBorder(
                    borderRadius: BorderRadius.circular(8),
                  ),
                ),
                child: const Text('Login', style: TextStyle(fontSize: 16)),
              ),
            ),
            TextButton(
              onPressed: () {},
              child: const Text('Forgot Password?'),
            ),
            Row(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                const Text("Don't have an account? "),
                TextButton(
                  onPressed: () {},
                  child: const Text('Sign Up'),
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```

Task 2: Create a Simple Home Screen

Objective:
Design a basic home screen displayed after login.

Expected UI Output :




```dart
import 'package:flutter/material.dart';

class SimpleListScreen extends StatelessWidget {
  const SimpleListScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        backgroundColor: Colors.blue,
        title: const Text('KindaCode.com'),
        elevation: 0,
      ),
      body: ListView.builder(
        itemCount: 13,
        itemBuilder: (context, index) {
          return Container(
            color: index.isEven ? Colors.grey[200] : Colors.white,
            padding: const EdgeInsets.all(16),
            child: Text('Item $index'),
          );
        },
      ),
    );
  }
}
```

Task 3: Navigate from Login screen to List screen

Objective:

Implement navigation from the Login screen to the List screen.

```
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) =>SimpleListScreen()),
  );
```

Task 4: Call the Login screen form Material App

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Login App',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: const LoginScreen(),
    );
  }
}
```

POST-LAB

1. **Enhance Navigation**
   a. Add a Logout button on the Home screen
   b. Navigate back to the Login screen using Navigator.pop()

2. Replace Layout Widget

   c. Replace Column with ListView in the Login screen
   d. Observe and explain the difference in scrolling behavior